AXON DIGITAL DESIGN

AxonNet objects

and

Axon Control Protocol

REV 1.4

COPYRIGHT © 2001 - 2012 AXON DIGITAL DESIGN BV

ALL RIGHTS RESERVED

NO PART OF THIS DOCUMENT CAN BE REPRODUCED IN ANY FORM
WITHOUT THE PERMISSION FROM AXON DIGITAL DESIGN BV.

NOTE:

This is a product under development. All information in this document is subject to change without notice.

Contact Axon Digital Design for more information on release dates.

Revision 0.2

- Error codes updated / Minor changes in text

Revision 0.3

- Object description added

Revision 0.4

- Object Type ID 2 is changed to "IP_Address" Object
- Type ID of "Long" Object is changed to 9
- "Byte" Object is added (Type ID = 10)
- General Refresh Flag in Access property is removed
- Minor change in ACP header (mtype can be 2 with announcements)

Revision 0.5

- Author changed to christiaan.simons@axon.tv
- The object Part 2 protocol is now named 'AxonNet objects'. This makes clear the difference between the ACP UDP wrapper protocol, and the actual structure of management information.
- Removed Ethernet / UDP / IP / ACP mapping diagram. Not useful and fixed offsets are not true if e.g. a timestamp ICMP option is used.
- Minor change in ACP header. Mcode field depends on mtype field, but it belongs to the objects.
- Changed introduction.
- Added TOC

Revision 0.6

- Added SPF errors to AxonNet error table.

Revision 1.0

- Bumped revision for the purpose of the AxonNet SDK release.

Revision 1.1

- Fixed some typos, updated copyright notice years.
- Mentioned transactions must be serialized to achieve flow control.
- Clarified object location discovery / user configuration requirement.

Revision 1.2

- Updated Alarm object definition with missing 'off' string.

Revision 1.3

- Corrected file object number of properties from 6 to 5
- Added reserved object type 11

Revision 1.4

- Updated copyright notice years
- A group can have up to 255 objects, not 256
- an object can have max 255 properties, not 256
- reuse same MTID when retransmittingclarification of the example timeout strategy
- swapped description of Alarm and File Objects, now all objects appear in order of increasing Object Type.
- Add Appendix with recommended practices.
- various small clarifications and fixes
- Add ACP (TCP version)

Table of contents

Contents

Axon Control Protocol	5
AxonNet objects	5
ACP Description	7
ACP Port Number	7
IP / UDP issues	7
ACP Message Types	8
ACP Header	9
Timeouts	12
ACP Announcements	13
ACP Reply Message Matrix	14
Object Groups	17
Object details	19
Objects by Type	21
The Root Object	21
The Integer Object	22
The IP Address Object	22
The Float Object	23
The Enumerated Object	23
The String Object	24
The Frame Status Object	24
The Alarm Object	25
The File Object	26
The Long Object	26
The Byte Object	27
Methods	28
AxonNet error codes	29
Method Overview	29
AxonNet - ACP mapping	32
Implementation Assistance	33
Further Reading	33
Appendix: Recommended Practices	34
ACP Driver Forward and Backward Compatibility	34

Introduction

This document describes the two protocols first introduced in the **Synapse** product range. The protocol described in part one is the Axon Control Protocol (ACP) and the second part describes the AxonNet object access protocol.

Axon Control Protocol

The ACP is a wrapper protocol for the UDP or TCP transport protocol. This thin layer was added to make multiple cards accessible using one IP address and port number. The ACP layer gives transparent access to the AxonNet objects.

The procedure for finding ACP servers on a network is described in the SDK. Only a preliminary version is available now.

AxonNet objects

AxonNet objects are the structure of management information. This part describes how **objects** can be accessed by using **methods** and defines the binary structure for these objects. Each Synapse card holds a number of objects. Objects are divided in **object groups** and each object has an **object ID** number. Note the object ID numbers between card software revisions are subject to change.

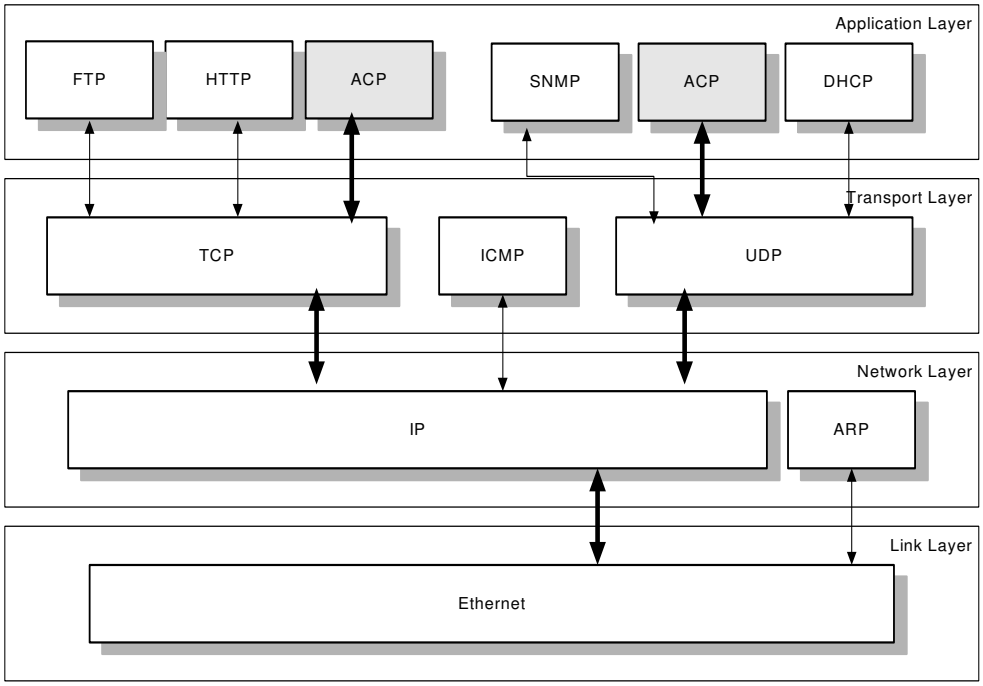
Methods are executed using an ACP **transaction** from a client on a remote server. The sequence of a client invoking a method (request) and a card returning a value (reply) is called a transaction. Changes on objects can be followed by listening to ACP **announcements**. These announcements can be generated by an ACP server and are sent to all clients connected to the network.

Alarm objects can generate a spontaneous Alarm announcement or 'Event'. These Alarms are used to **inform** ACP clients about a changing condition on a card.

PART 1 - ACP

This chapter describes the Axon Control Protocol (ACP). ACP is designed for control and monitoring applications for the Axon Synapse system. ACP uses IP for its network layer and UDP for its transport layer. The physical layer is 10Base-T or 1000Base-T Ethernet.

The following diagram shows the route (thick lines) that ACP packets travel through the TCP/IP Protocol Suite.



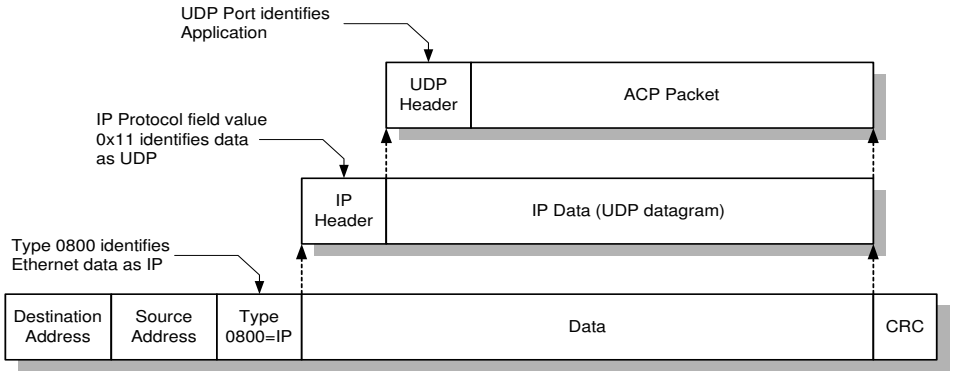
TCP/IP Protocol Suite

UDP is a connectionless protocol that does not guarantee packets delivery. To make UDP work in a reliable way, ACP adds transaction ID numbers and a timeout and retransmission mechanism. Flow control is achieved by serializing the transactions.

ACP adds a message subaddress for addressing multiple cards using one IP address. The message type field is added for discriminating between incoming and outgoing transactions.

ACP Description

ACP data packets (or segments) are carried in UDP or TCP datagrams. UDP/TCP datagrams are carried in IP datagrams and IP datagrams are carried in Ethernet frames:



ACP Encapsulation

ACP Port Number

IANA (Internet Assigned Numbers Authority) has assigned port number **2071** to the ACP protocol:

acp-port	2071/tcp	Axon Control Protocol
acp-port	2071/udp	Axon Control Protocol

IP / UDP issues

Because the ACP payload is relatively small (<= 134 bytes), IP fragmentation and IP options is not supported in ACP.

The maximum size of the IP datagram is 169 bytes (20 bytes IP Header + 8 bytes UDP Header + 7 bytes ACP Header + 134 bytes ACP data).

The UDP checksum **must** be implemented because this is the only test for ACP data integrity.

ACP Message Types

ACP defines two classes of messages for communication:

1. **Broadcast Messages:**

   Servers as well as clients can initiate **Broadcast Messages**. Broadcast Messages are **Subnet Broadcasts** and are only received by the nodes within a subnet. Broadcast Messages are never acknowledged thus data delivery is not guaranteed.

   A Broadcast initiated by a server is called an **Announcement**. Announcements are generated on a server to inform all clients on a subnet of changes on control values or status items.

   If obtaining status is critical, transaction messages must be used instead (polling).

   The only broadcast message clients are allowed to invoke a **Broadcast getValue(FrameStatus) Method**. This will trigger all rack controllers on the subnet to reply with its FrameStatus value to that client.

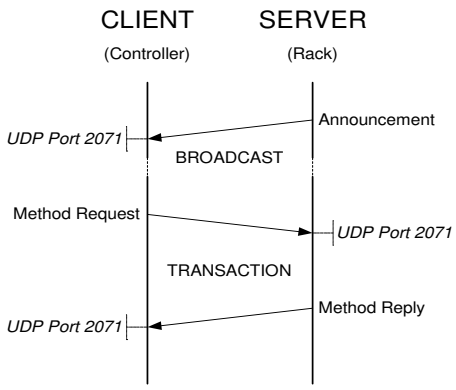
2. **Transaction Messages:**

   These are unicast messages sent between controllers (clients) and racks (servers) in order to perform transactions such as changing parameters or requesting status information. Two basic messages are defined: **Request Messages** and **Reply Messages**.

   If a client wants to perform some action on a server, it must first send a **Request Message** to that server. A server will respond by sending a **Reply** back to the client. Optionally, the server may announce (broadcast) the Reply on the subnet or additionally direct the Reply to a **Gateway**.

   If, for some reason, an error occurred during the handling of a Request Message, a server will return a special Reply Message, called a **Reply Error Message**, which contains an error description code.

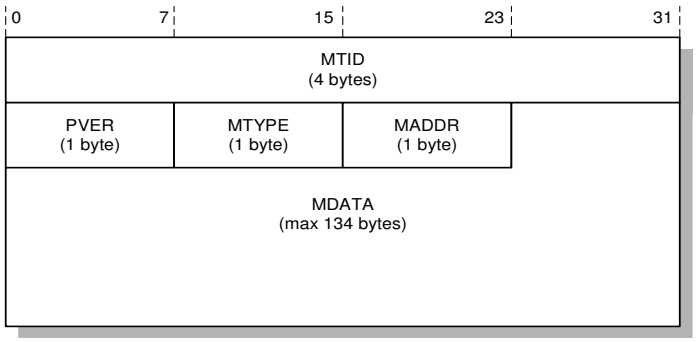
The following diagram shows the flow of possible transactions:



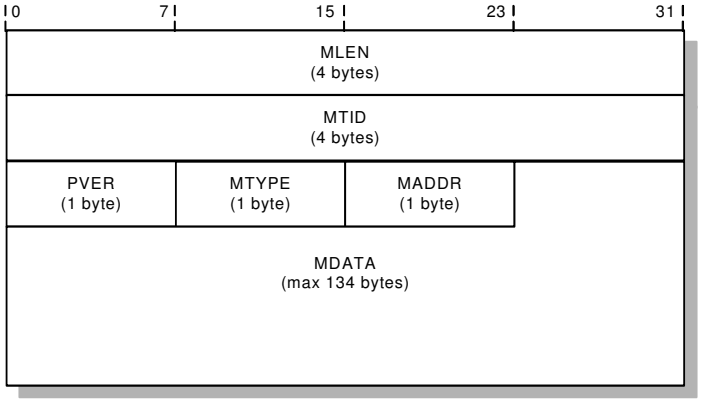
ACP Communication Flow

ACP Header

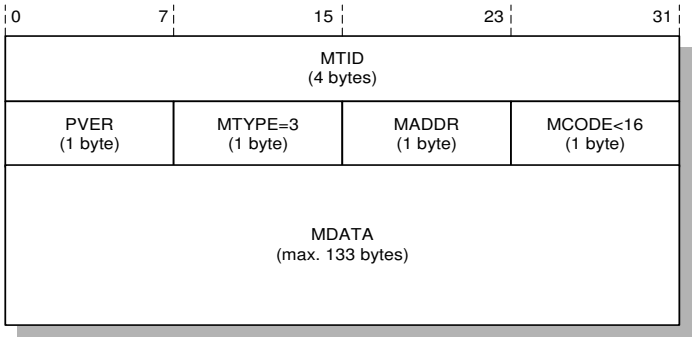
An ACP packet (embedded in a UDP datagram) utilizes a 7 bytes header and variable length data (maximum 134 bytes). When TCP is used 32 bits are added to the package length. This MDATA field holds the AxonNet object payload described in Part 2. The format of an ACP packet is as follows:



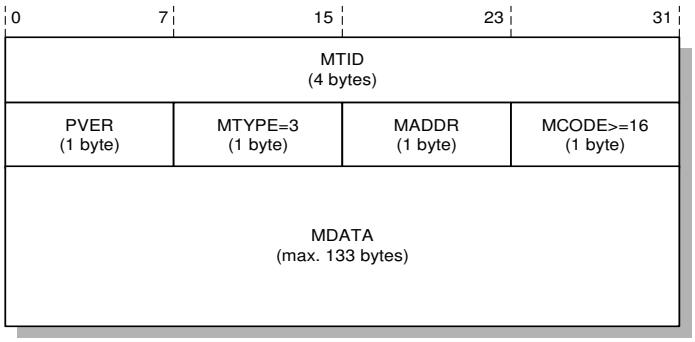
ACP message (UDP)



ACP message (TCP)



ACP error message



AxonNet error message

MLEN: MLEN is a 32-bit number (MSB) specifying the length of the remaining message in bytes (starting with MTID, so

MLEN must be > 8 due to the sizes of the other fields). (Only used for TCP version)

MTID: MTID is a 32-bit number that is used either as a Transaction Identifier or as a Broadcast Identifier.

Transaction Identifier:

A client randomly generates an initial MTID (Message Transaction ID) at power-up. A MTID must not be zero. It is incremented by the client when initiating a new transaction (sending a Request Message) and returned by a server who responds to that transaction (sending a Reply Message).

When the client receives a Reply Message it compares the MTID returned by the server with the MTID of the Request Message sent. If they don't match, the client discards the message and waits for the next message from the server. Each time the client sends a new Request Message it increments the MTID. If the client retransmits a previously sent Request Message (because it has not received a Reply Message), the MTID should not change.

Broadcast Transaction Identifier:

With Broadcast Messages, the Message Transaction ID is always set to zero.

PVER: PVER, the Protocol Version Number, is an 8 bit unique number that indicates the version of the ACP protocol. The current version is 1.

MTYPE: This is a one-byte field indicating the type of message. The following message types have been defined:

MTYPE	Description
0	Announcement Message
1	Transaction Request Message
2	Transaction Reply Message
3	Transaction Error Reply Message

MADDR: MADDR (Message Sub Address) is a one-byte field. With a Request Message, this field contains the destination sub address in a Synapse frame (e.g. a card or the rack controller). With a Reply Message or a Broadcast Message, this field contains the source sub address. The address should be in the range 0..31. The rack controller itself is at address 0.

MCODE: Dependent on the Type of Message, this field contains an ACP or AxonNet error.

If MYPE < 3 this field is the first byte of the AxonNet payload.

If MTYPE is 3 and MCODE < 16 this is an ACP error message and indicates a transport error. AxonNet object errors are described in Part 2 of this document.

	Error Description
0	Undefined Error
1	Internal Bus Communication Error Occurred **Communication to the addressed card fails. (Checksum error or data length error). The transaction is cancelled.**
2	Internal Bus Timeout Error Occurred **Communication to the addressed card cannot be established. (The card is not present). The transaction is cancelled.**
3	Transaction Timeout Occurred **Communication to an addressed card is established, but the card does not generate a Reply within the Transaction Timeout Time. The transaction is cancelled.**
4	Out of Resources (TCBs)
4 < mcode < 16	**Not Defined**

MDATA: The MDATA field contains the Message Data. This is a variable length data field that may contain the Method Arguments (Request Message and Broadcast Messages) or the Method Return values (Reply Message). The maximum length of a data block is 134 bytes (133 for error replies).

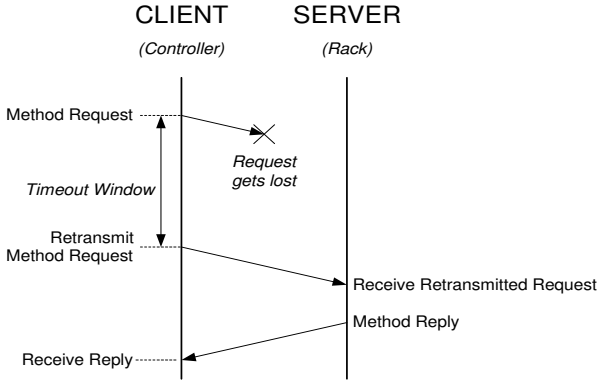
Timeouts

Applications using ACP should implement a timeout mechanism. The mechanism should operate on the client side and only on transmitted request messages. It is the responsibility of the application developer to implement a solid timeout mechanism. A possible mechanism is described below:

If a client transmits a Request Message (starts a transaction), it starts a timeout counter. If the transmitted Request Message or Reply Message gets lost, no reply will arrive from the server and the timeout counter will timeout.

If the request that timed out is a set request, it is highly recommended to do a get request of the property to see if it has been changed. If not the request should be retransmitted.

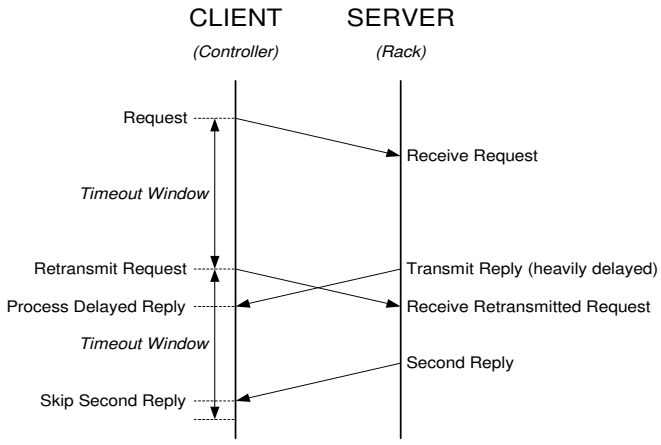
If the request that timed out is a get request the client may retransmit the original Request Message (with the same MTID).



Retransmission Flow

To avoid congestion, an exponential back-off algorithm can be used.

The client is responsible for checking the MTID of the reply. Because MTIDs are unique unknown replies can be discarded and should not appear.



Avoiding Duplicate Replies

ACP Announcements

When multiple clients are connected to a Synapse network, it is important that a change made by one client to a card is announced to all other clients on the network. Announcements are generated in the following situations:

1. **A change occurs in a frame:**

- A card is inserted
- A card is removed
- A card is powering up
- A card is in an error state
- A card is in Boot Mode

2. **A change occurs in a card's control item:**

   A client has changed a control parameter. The new value of the control parameter is "announced" to all other clients. The source of this announcement is the rack controller. The controller listens to the replies of setValue, setDefault, setInc and setDec methods and announces it on the network.

3. **A change occurs in a card status item:**

   The status of some property on a card has changed. The new status value is "announced" to all clients. The source of this announcement is the card.

4. **An Alarm is generated on a card:**

   An Alarm event is generated on a card or frame. This event is "announced" to all clients. The information transmitted by such a message is volatile. It cannot be retrieved from the system after transmission.

Alarm Announcements can be individually enabled by setting their priority value to a value greater than zero. Both Status and Alarm Announcements can be globally disabled on the card by setting the first Alarm object (called 'Announcements') to zero.

The broadcasting of announcements can be enabled or disabled on the Synapse rack controller. Because Announcements are only broadcasted within the sub-net, the rack controller also supports forwarding Announcements to one IP Address (which may be on a different sub-net).

ACP Reply Message Matrix

	mtid	mtype	mcode	Object Group	Object ID	Description
Announcement Card Identity	0	0	0	1	X	Triggered by event on Card
	0	2	1,2,3,4	1	X	Transaction Announcement
Announcement Card Control	0	0	0	2	X	Triggered by event on Card

	0	2	1,2,3,4	2	X	Transaction Announcement
Announcement Card Status	0	0	0	3	X	Triggered by status change on Card
Announcement Card Event	0	0	0	4	X	Triggered by event on Card (= Event Message)
	0	2	1,2,3,4	4	X	Transaction Announcement
Announcement Frame Status	0	0	0	6	0	Triggered by status change on Frame
Transaction Reply	> 0	2	X	X	X	Generated after invoking a Request
Transaction Error Reply	> 0	3	Error Code	X	X	Generated after invoking a Request and an error occurred on the server

Reply Message Matrix

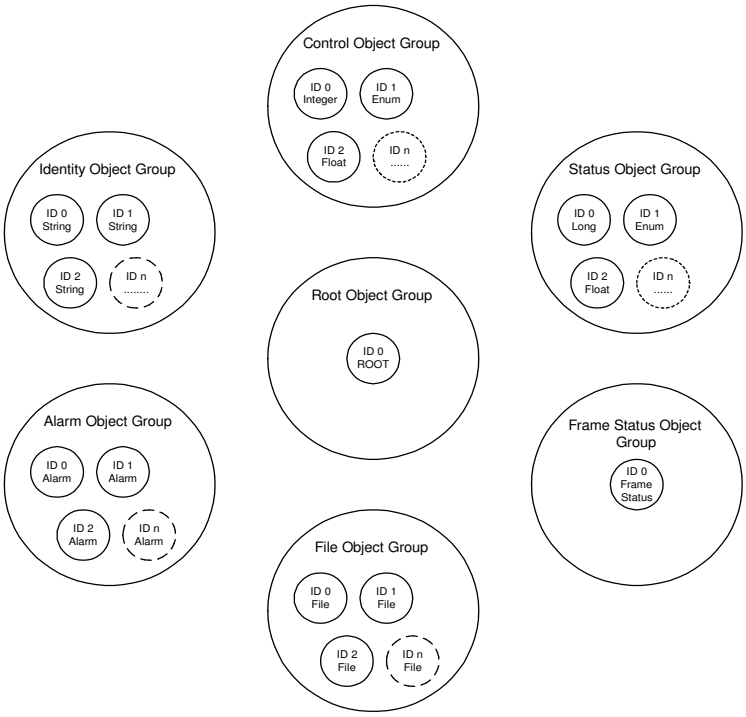
Note that mcode 1, 2, 3, 4 refers to the method ID's as specified later on. Methods 0 and 5 are "get" methods and can never result in announcements. Mcode 0 is reused to indicate a card event.

Note also that the File Object Group will never generate an announcement.

PART 2 – AxonNet objects

An Object is an entity on a Card that has a value. Examples are Video Gain or Input Status. Depending on the access rights of an Object, the value can be read, written, incremented, decremented or set to default.

Objects are divided into Objects Groups. Each Object Group has a Group ID and each Object in a Group has an Object ID. The Group ID and the Object ID are byte sized indexes. Objects are not guaranteed to remain at the same index location for different card firmware revisions. **AxonNet client applications must implement either a runtime discovery of object IDs or have runtime user configurable object IDs.**



Object Group / Object relation

The Root Object holds information of the number of Objects in each Object Group. This object can be read first to determine the available objects per group. This is an essential part of the runtime object ID discovery process.

For a detailed description of the implemented Objects on a card, see the corresponding card manual.

AxonNet client applications may not use hard coded object ID's for the Control, Status or Alarm object groups.

Object Groups

To ease the creation of user applications and interfaces, Objects are divided into Object Groups. Each Object Group has an Object Group ID and each Object Group may contain up to 255 Objects. Each Object can be accessed through its Group ID and Object ID.

The following Object Groups are defined:

Group ID	Description
0	Root Object Group
1	Identity Object Group
2	Control Object Group
3	Status Object Group
4	Alarm Object Group
5	File Object Group
6	Frame Object Group

- **Root Object Group**

  Each Card has one Root Object Group and the Root Object Group contains one Root Object. The Root Object contains information about the number of Objects in each Object Group and the mode of operation of the microprocessor.

- **Identity Object Group**

  This Object Group contains several Objects that contain information about the Identity of a Card. The following Objects are mandatory for the Identity Objects Group (more Objects may be present):

Mandatory Identity Objects	
Object ID	**Description**
0	Card Label
1	Card User Definable Label
2	Card Description
3	Card Software Revision
4	Card Hardware Revision
5	Card Product Code
6	Card Serial Number
7	Card ID
8..255	Additional Objects

- **Control Object Group**

  This Group contains Objects that control parameters of a Card.

- **Status Object Group**

  This Group contains Objects that inform about the status of a Card.

- **Alarm Object Group**

  The Alarm Object Group can only contain Alarm type Objects. With Alarm type Objects a client may set priorities on Alarm Events.

- **File Object Group**

  This Group can only contain File type Objects. The following Objects are mandatory in the File Objects Group:

Mandatory File Objects	
Object ID	**Description**
0	Microprocessor Firmware
1	Parameter Table
2..255	Additional File Objects

  Note that additional File Objects may be present in this group.

- **Frame Object Group**

  This Group contains only one Frame Status Object. At this moment only the rack controller will have a Frame Object Group.

Object details

An Object can operate on a specific data type. There are several Object types defined for data types like integer values, enumerated values, strings and floating point values. Furthermore a few specific objects were defined which are used for files, frame content and object administration.

An Object contains several Properties. The exact number of properties depends on the Object Type but three properties are mandatory:

- **Property Object Type**

  The Object Type Property (byte value) denotes the Object Value type:

Object Type ID	Description
0	Root Type
1	Integer Type
2	IP Address Type
3	Floating Point Type
4	Enumerated Type
5	String Type
6	Frame Type
7	Alarm Type
8	File Type
9	Long Type
10	Byte Type
11	Reserved
12..255	Undefined

- **Property Number Of Properties**

  This Property (byte value) holds the number of Properties that reside in an Object.

- **Property Access**

  This Property (byte value) denotes the Access rights to an Object:

Access Property	
Bit 0	Read Access (0=no / 1 = yes)
Bit 1	Write Access (0=no / 1 = yes)
Bit 2	SetDefValue Access (0 = no / 1 = yes)
Bit 3	not used
Bit 4	not used
Bit 5	not used
Bit 6	not used
Bit 7	not used

  Some Objects are linked to other Objects on a Card. For example, when changing the format of a decoder card from 625 lines to 525 lines, the H-Phase value may be truncated.

A number of Properties are common to several Objects:

- A **Label String** holds an Item description (Video Gain, H-Phase, Format….). The maximum string length is 16. This is a null terminated string. Note that this label can be used **to locate an object by name**. This is preferable if the software needs to cope with card firmware revision changes.
- A **Unit String** holds a value postfix description (uS, secs, pixs, dB…). The maximum length of a string is 4. This is a null terminated string.
- The **Default Value** is fixed value, defined by Axon. It can be used to reset a value.
- The **Minimum Value** and **Maximum Value** properties are the limits of a value. When exceeding a limit, the Card will return either the Maximum Value (in case of a positive number) or the Minimum Value (in case of a negative number).
- The **Default Step Size** denotes the increment value for an Object numeric value. Most times the Default Step Size will be 1.

Notes:

- All integer values, long values and float values are transmitted with the MSB first.
- A Byte is an unsigned 8-bit value.
- An Integer is a 16-bit signed value, two's complement.
- A Long is a 32-bit signed value, two's complement.

- All strings are null-terminated. The Maximum String Length does not include the null-terminator.
- Floats are single precision (binary32) values encoded according to IEEE 754
- Objects are limited in size to 131 bytes. The total size of an ACP message is 141 bytes. The header uses up 7 bytes. After that is the method (in the MCODE field), group ID and object ID, leaving up to 131 bytes in the payload.

Objects by Type

Currently 11 Object Types are defined. The number of Object Types may be extended in the future while keeping backwards compatibility.

The shaded Properties indicate the Object Value.

The Root Object

The Root Object is used to retrieve information from a Card about the number of Objects in each Class. In addition, the value of this Object accesses the mode of operation of the microprocessor and is used for programming the microprocessor firmware.

Root Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	0
1	Number of Properties	byte	9
2	Access	byte	-
3	Boot Mode	byte	-
4	Number of Identity Objects	byte	-
5	Number of Control Objects	byte	-
6	Number of Status Objects	byte	-
7	Number of Alarm Objects	byte	-
8	Number of File Objects	byte	-

- The **Boot Mode** property in the Root Object is used when programming the microprocessor firmware. The Boot Mode field in the Root Object of a Card is 0 when in regular operation.
- The **Boot Mode** property is the value property of the root object. So when invoking the getValue method, the Boot Mode value is returned.

The Integer Object

The Integer Object works on integer (16 bit signed) values.

Integer Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	1
1	Number Of Properties	byte	10
2	Access	byte	-
3	Value	integer	-
4	Default Value	integer	-
5	Default Step Size	integer	-
6	Minimum Value	integer	-
7	Maximum Value	integer	-
8	Label String	string	-
9	Unit String	string	-

The IP Address Object

The IP Address Object works on 32 bit signed values.

IP Address Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	2
1	Number Of Properties	byte	10
2	Access	byte	-
3	Value	32 bits unsigned	-
4	Default Value	32 bits unsigned	-
5	Default Step Size	32 bits unsigned	-
6	Minimum Value	32 bits unsigned	-
7	Maximum Value	32 bits unsigned	-
8	Label String	string	-
9	Unit String	string	-

The Float Object

This Object Type works on floating point numbers. The format used is standard IEEE 754 single precision (4-byte) numbers.

Float Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	3
1	Number Of Properties	byte	10
2	Access	byte	-
3	Value	float	-
4	Default Value	float	-
5	Default Step Size	float	-
6	Minimum Value	float	-
7	Maximum Value	float	-
8	Label String	string	-
9	Unit String	string	-

The Enumerated Object

This Object works on enumerated values and contains a list of named values (items).

Enumerated Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	4
1	Number Of Properties	byte	8
2	Access	byte	-
3	Value	byte	-
4	Number of Items	byte	-
5	Default Value	byte	-
6	Label String	string	-
7	Item String List	array of bytes	-

- The **Number of Items** property holds the number of Enumerated Items, contained in the Item String List.
- The **Items String List** contains the comma delimited enumerated Item Strings. The Items String List is null-terminated. The length of an object is limited to 131 bytes (see object details). Thus label and items are max 125 bytes long, including the two null-terminators.

The String Object

With this Object, null-terminated strings can be accessed.

String Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	5
1	Number Of Properties	byte	6
2	Access	byte	-
3	Value	string [MaxLen]	-
4	MaxLen	byte	-
5	Label String	string	-

- The **MaxLen** Property defines the maximum length a string may have. This does not include the null-terminator.
- The length of an object is limited to 131 bytes (see object details). Thus label and value are max 127 bytes long, including the two null-terminators

The Frame Status Object

This Object is used to retrieve information from a frame about the status of every slot in the frame. There is only one Frame Object in the Frame Status Group. The Object Value is also used with Announcements. This Object is Read Only.

Frame Status Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	6
1	Number Of Properties	byte	4
2	Access	byte	1
3	NumOfSlots + Slot Status Array	array of bytes	-

- The **NumOfSlots** Property holds the number of slots of a Frame, including the rack controller. So, NumOfSlots is 19 for the SFR18.
- Each **Slot Status Array** element is a byte value. Possible element values are:

Slot Status	
0	No Card
1	Card Power Up
2	Card Present
3	Card Error
4	Card Removed
5	Card Boot Mode

The Alarm Object

The Alarm Object is triggered by a condition on a Card. The client can enable or disable and set the Priority of the Event.

Alarm Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	7
1	Number Of Properties	byte	7
2	Access	byte	-
3	Enable / Priority	byte	-
4	Tag	byte	-
5	Label String	string	-
6	Event Message On String Event Message Off String	string [max 32] string [max 32]	-

When the **Priority** Property is 0, Alarms are disabled.

- The **Tag** Property is a fixed byte value assigned by Axon.
- This **Event Message String** property holds the null-terminated string that is send when the Alarm Object is triggered.
- An Alarm Message has the following format:

  <ObjectGroup><ObjectID><Priority><Tag><Event Message String>

The File Object

This Object is used to Access Files on a Card.

File Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	8
1	Number Of Properties	byte	5
2	Access	byte	-
3	Number of Fragments	integer	-
4	File Name	string[max 16]	-
5	Fragment ID Fragment Length Fragment Data	integer byte array of bytes[0..127]	-

File Objects allow reprogramming of firmware. However reprogramming is a relatively complex matter (using encryption), which must be left to dedicated programs, supplied by Axon, such as SynapseSetup or Cortex.

For normal control and monitoring applications, the implementation of File Objects is not needed.

Note: When invoking a getObject on this Object, the Fragment Property is **not** returned.

The Long Object

The Long Object works on long (32 bit signed) values.

Long Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	9
1	Number Of Properties	byte	10
2	Access	byte	-
3	Value	long	-
4	Default Value	long	-
5	Default Step Size	long	-
6	Minimum Value	long	-
7	Maximum Value	long	-
8	Label String	string	-
9	Unit String	string	-

The Byte Object

The Byte Object works on 8 bit unsigned values.

Byte Object			
Property Index	**Property Description**	**Format**	**Fixed Value**
0	Object Type	byte	10
1	Number Of Properties	byte	10
2	Access	byte	-
3	Value	byte	-
4	Default Value	byte	-
5	Default Step Size	byte	-
6	Minimum Value	byte	-
7	Maximum Value	byte	-
8	Label String	string	-
9	Unit String	string	-

Methods

Objects, or Object Values, are accessed using Methods. The following Methods are defined:

Method	ID	Arguments	Returns
getValue	0	none	Value
setValue	1	Value	Value
setIncValue	2	none	Value
setDecValue	3	none	Value
setDefValue	4	none	Value
getObject	5	none	Property Value List

Methods Overview

Not all Objects support all Methods. If an invoked Method is not supported on an Object, an Error Reply is returned. See the following table:

	Root	Integer Long Float Byte IPAddr	Enum	String	Alarm	File	Frame
getValue	x	x	x	x	x	x	x
setValue	x	x	x	x	x	x	
setIncValue		x	x		x		
setDecValue		x	x		x		
setDefValue		x	x	x	x	x	
getObject	x	x	x	x	x	x	x

Supported Methods for each Object Type

Integer values, Long values and Float values use the MSB first format for arguments and return values.

AxonNet error codes

MCODE	Error Description
16	Object Group does not exist
17	Object Instance does not exist
18	Object Property does not exist
19	Object Property has no Write Access
20	Object Property has no Read Access
21	Object Property has no Set Default Access
22	Object Type does not exist
23	Illegal Method
24	Illegal Method for this Object
24 < mcode < 32	**Not defined.**
32	File Error
32 < mcode < 39	**Not defined.**
39	SPF file doesn't meet constraints.
40	SPF buffer full, retry fragment later.

Method Overview

getValue Method

Syntax: *value* getValue(*objectgroup, objectid*)

Description: Used to retrieve the current value of an Object

Arguments: ObjectGroup, ObjectID

Returns: The current value of the Object

Method ID: 0x00

setValue Method

Syntax: *value* setValue(*objectgroup, objectid, value*)

Description: Used to set the value of an Object.

Arguments: ObjectGroup, ObjectID, Value

Returns: The modified value of the Object

Method ID: 0x01

Notes: The argument value is range checked and truncated if necessary on the card before used as the Method return value.

setIncValue Method

Syntax: *value* setIncValue(*objectgroup, objectid*)

Description: Used to increment the (numeric) value of an Object.

Arguments: ObjectGroup, ObjectID

Returns: The modified value of the Object

Method ID: 0x02

Notes: The value of the Object is incremented. If the Object Value already reached its maximum value, then the maximum value is returned.

This Method does not operate on all type of Objects. If invoked on a non-numeric Object type, an Error Reply is returned.

setDecValue Method

Syntax: *value* setDecValue(*objectgroup, objectid*)

Description: Used to decrement the (numeric) value of an Object.

Arguments: ObjectGroup, ObjectID

Returns: The modified value of the Object

Method ID: 0x03

Notes: The value of the Object is decremented. If the Object Value already reached its minimum value, then the minimum value is returned.

This Method does not operate on all type of Objects. If invoked on a non-numeric Object type, an Error Reply is returned.

setDefValue Method

Syntax: *value* setDefValue(*objectgroup, objectid*)

Description: Used to set the Object to its default value.

Arguments: ObjectGroup, ObjectID

Returns: The modified (and default) value of the Object

Method ID: 0x04

Notes: This Method does not operate on all type of Objects. If invoked on an -unsupported Object type, an Error Reply is returned.

getObject Method

Syntax: *value* getObject (*objectgroup, objectid*)

Description: Used to retrieve all property values of the Object in one sequence.

Arguments: ObjectGroup, ObjectID

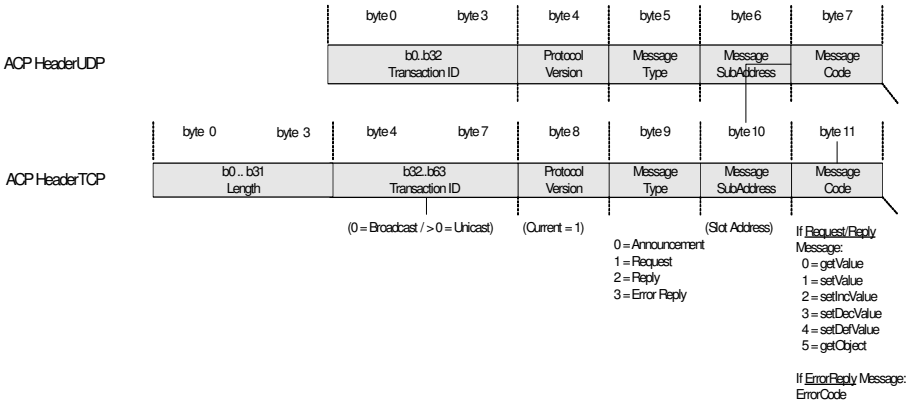
Returns: A list of property values. See Object Type descriptions for the order and alignment.

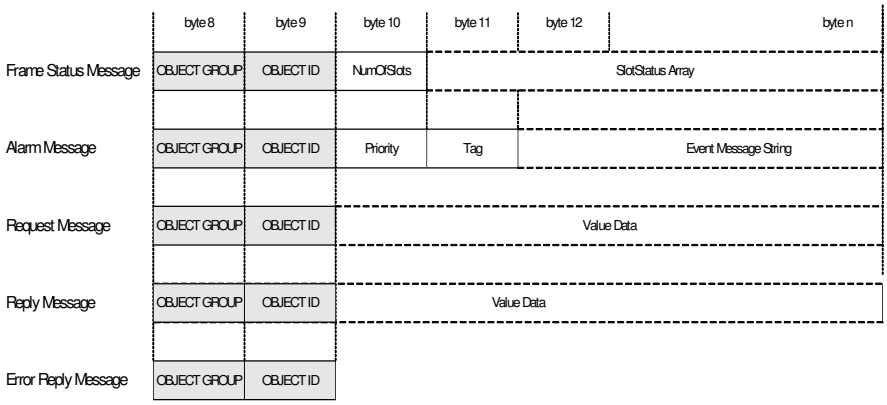
Method ID: 0x05

Notes: Property Values are transmitted starting from Property ID 0.

This Method operates slightly different with File Objects.

AxonNet - ACP mapping





Implementation Assistance

A Source Development Kit (SDK) is under development.

Further Reading

Internetworking with TCP/IP Principles, Protocols, and Architecture

Douglas E.Comer

Prentice Hall

TCP/IP Illustrated

W.Richard Stevens

Addison-Wesley

RFCs can be obtained from http://www.rfc-editor.org/

Information about Ethernet can be found at http://www.ieee802.org/

Appendix: Recommended Practices

This section of the document describes the recommended practices when writing a software driver for ACP.

ACP Driver Forward and Backward Compatibility

Each ACP controlled product has a defined number of objects in the control, status and alarm groups. These objects are defined for a specific software version of each product, details of which can be issued from Axon's support department.

As the development cycle of these products continue features are added. This, in turn, leads to new objects being potentially added to one or more of these groups.

These new objects will be added in the most logical position within the group, thus leading to the potential issue of the same object changing its **object ID** over different software revisions, as mentioned previously in the document.

For this reason in order for software drivers to maintain both forward and backward compatibility with ACP controlled products it is recommended that the software uses the objects **Label String** rather than its **object ID** to identify the object to be accessed.

The **Label String** will **ALWAYS** be unique within the products object group to which it belongs. Using this field as the unique identifier will give the benefit of being able to control/monitor different revisions of the same product at the same time and also similar types of product (provided that the **Label String** fields remain the same).

As ACP is driven using the **object Group** and **object ID**, it is recommended that on initiating control of a specific instance of an ACP product all objects for that group are read using the **getObject** method so that an internal mapping table can be formed to correlate the objects **Label String** to its **object ID.** From this point on the client software should ideally refer to an Object by its **Label String** and only when sending/receiving a message to/from the device referring to its **object ID.**